

Project Final Report — Dungeon Grid Arena

Kerem Inci 德珍

2025080281

C++ Final Project

Source code: <https://github.com/xDKRMx/Dungeon-Grid-Arena.git>

This document **Project Final Report** is divided into the three requested by the instructor:

- **Part 1 — Project Proposal** (motivation, scope, schedule)
- **Part 2 — Requirements Analysis Report** (functional and non-functional requirements, operational restrictions)
- **Part 3 — Design Specification** (architecture, class design, sequence diagrams, file I/O, algorithms)

Part 1 — Project Proposal

1.1 Title and one-line description

Dungeon Grid Arena — a turn-based, chess-inspired dungeon rogue-like written in C++17. The canonical experience is the Raylib graphical build (full HUD, procedural SFX, streaming OGG music, Death Dungeon visual mode). A minimal ASCII console build sits behind the same `IRenderer` interface as proof that the renderer abstraction is loosely coupled; every screenshot, demo video, and gameplay walkthrough in the submission targets the Raylib build.

1.2 Motivation and topic justification

The course offered a free choice of topic, with chess and the Raylib example library suggested as anchor ideas. I chose to combine both:

- **Chess** is the inspiration for the enemy taxonomy. Each enemy archetype borrows a chess piece's movement / attack rule:
 - Rook fires along straight rows and columns.
 - Bishop fires along diagonals.
 - Queen fires along both.
 - Plus a Melee, a Fast 2-step variant, and a Boss.This gave the game a coherent design language without copying the chess rules wholesale.

- **Raylib** is the inspiration for the graphical renderer. The reference samples on the Raylib website demonstrate windowing, primitive drawing, audio, and streaming music. I wanted to use all of those so the project could showcase a real graphical front-end without falling back on console output for the demo.

Crucially, the gameplay was designed so that **every optional knowledge point** of the rubric — inheritance, polymorphism, templates, hand-rolled linked lists, file I/O — has a *reason* to exist:

- Inheritance: Entity → Player + Enemy → 6 chess-style archetypes.
- Polymorphism: virtual `glyph()` for rendering, virtual `canAttackPlayer` and `decideAction` for AI, virtual `applyTo` for item effects, virtual `activate` for abilities.
- Templates: `Grid<T>` is shared between the dungeon map and the BFS visited buffer; `RingBuffer<T>` underpins fixed-capacity FIFO usage.
- Hand-rolled linked list: `EventLog` shows the most recent N gameplay messages on the HUD — the data structure is literally visible to the player every turn.
- File I/O: a transactional save/load with full staging-then-commit, plus a sorted top-N leaderboard appended to a text file on death.

This way the OOP and data-structure features are not academic decoration — they are load-bearing parts of the game.

1.3 Group composition and responsibilities

Member	Responsibilities
Kerem Inci	Solo project. All design, implementation, testing, and documentation.

1.4 Initial schedule

The proposal was filed on May 7. Work proceeded against the following milestones:

Date	Milestone
Jun 1- 4	Project proposal, topic, target rubric items, deliverable list determined. Requirements analysis and design specification. Core layers implemented: <code>core</code> , <code>world</code> , <code>entities</code> , <code>combat</code> . Test suite stood up. Systems layer: turn manager, abilities, waves, save / load, ASCII smoke-test renderer.
Jun 5 – 7	Raylib renderer, audio synthesis, gameplay polish, single-panel wave-clear menu.
Jun 8	Final submission package assembled and uploaded.
Jun 10	Live demonstration session.

Part 2 — Requirements Analysis Report

2.1 System purpose

Dungeon Grid Arena is a single-player, turn-based dungeon roguelike. The player controls one hero on a procedurally generated grid, faces waves of chess-style enemies, collects items, spends earned gold in a shop, and tries to reach as deep a wave as possible before dying. The game is intended to demonstrate idiomatic, well-organised, well-tested C++17 in a real running program rather than as fragmentary code samples.

2.2 Functional requirements

The full requirements catalogue below enumerates the 30 requirement groups. Each group expands into 3 – 8 verifiable sub-clauses.

ID	Requirement summary
R1	Encapsulation: every gameplay value lives behind member functions of an owning class.
R2	Multi-file structure: every non-trivial class has a <code>.h</code> + <code>.cpp</code> pair.
R3	File I/O: leaderboard, save / load, optional config file.
R4	Inheritance & polymorphism: 4 abstract bases (<code>Entity</code> , <code>Item</code> , <code>Ability</code> , <code>IRenderer</code>).
R5	Templates: <code>Grid<T></code> and <code>RingBuffer<T></code> .

ID	Requirement summary
R6	Hand-rolled singly linked list: <code>EventLog</code> .
R7	Determinism: a fixed RNG seed reproduces an identical run.
R8	Strict layered architecture: lower layers may not include higher-layer headers.
R10	Five-phase turn loop (player → enemies → deaths → upkeep → counter / clear check).
R11	Movement: orthogonal walking, blocked walls, melee attack on enemy tile, walk-over pickup.
R12	Enemy AI: BFS pathing toward the player, <code>FastEnemy</code> with up to 2 sub-steps per turn.
R14	Chess-style enemy attack rules (rook / bishop / queen) gated by range and line of sight.
R15	Damage formula and clamped health.
R16	Player Fire: ammo, cooldown, projectile trace, miss-on-wall, range bonus from ranged weapon.
R17	Wave system: every 5th wave is a boss wave; per-wave enemy count growth.
R18	Boss enemy spawn rule and balance.
R19	Item pickups (potion, weapon, ammo, armor, treasure).
R20	Inventory: pickup is implicit on walk-over; weapons / armor apply on pickup.
R21	Abilities (Dash / Nova / Shield / Blink) with per-ability cooldowns.
R22	Charge meter feeding the Nova ultimate; meter fills with kills.
R23	Between-wave 3-card upgrade draft (free).
R24	Score formula: <code>wave × 100 + kills × 10 + treasure value</code> .
R25	Top-10 leaderboard persistence in <code>data/highscores.txt</code> .
R26	Save / Load with transactional commit; backwards compatible across format versions.
R27	Main menu (Resume / New / Load / High Scores / Settings / Quit), game-over screen.
R28	Renderer abstraction: at least one renderer; the project ships two.
R29	HUD content: HP, ammo, range, gold, wave, score, abilities, recent events, legend.
R30	Top-level game loop driven by a single <code>Game::run</code> .

In addition, four post-proposal **features** were added during the implementation phase based on playtest feedback:

Feature	Description
F1	Gold currency decoupled from Score — Treasure pickups feed both, but Gold is spendable.
F2	Paid Shop after each wave — three rotating items the player buys with Gold.
F3	Weakened free upgrade pool — the strongest cards moved into the Shop; new <code>BonusGold</code> filler.
F4	Settings menu with a master music volume slider that re-applies to streams live.

After the playtest round revealed the two-panel wave-clear flow was confusing, F2 and F3 were merged into a **single combined panel** with disabled-row navigation skipping for unaffordable shop entries.

2.3 Non-functional requirements

- **Performance:** 60 FPS on commodity Windows hardware in the Raylib build. SFX synthesis at startup completes under 200 ms. The ASCII smoke-test build is responsive on any 80×30 terminal.
- **Build cleanliness:** warning-free with `-Wall -Wextra` on all three build targets (Raylib, ASCII smoke-test, test runner). A warning is a defect.
- **Portability:** pure C++17; the only platform-specific code is the keyboard-read fallback in the smoke-test `ConsoleRenderer.cpp` (`_getch` on Windows vs. `tcsetattr` on POSIX).
- **Determinism:** a fixed seed reproduces the same map, the same item drops, the same enemy roll outcomes, and the same upgrade / shop drafts. The full run is reproducible.
- **Safety:** save / load is transactional. A corrupted save file can never partially overwrite the live `GameState`.
- **Memory hygiene:** every owned heap object is held by a `std::unique_ptr`. Ownership transfers are explicit `std::move`. No raw `new` / `delete` outside of test fixtures.
- **Graceful degradation:** missing `assets/normal_theme.ogg` or `assets/boss_theme.ogg` files do **not** crash the game; the affected wave plays only the procedural ambient pad.

2.4 Operational restrictions and input rules

The operational restrictions are documented in `USER_MANUAL.md` at chapter 16. They are enforced by the code and applicable to every input. The most important are:

- Movement is one tile per key press, orthogonal only.
- Fire requires a direction key (`W/A/S/D` or arrows). `R` cancels.
- Fire is rejected on three conditions, each with an instant on-screen notice and **no turn cost**: no ammo, on cooldown, invalid direction.
- Walking into a wall or out of bounds cancels the move with a log line and **does not consume the turn** — the player tries again.
- Picking up an item is implicit on walk-over.
- `Ctrl+S` saves; `Q` returns to the main menu without saving.
- Game Over input accepts only `Space` / `Enter` / `Q` / `Esc`.
- Settings volume changes are in-memory only and reset on process restart.
- The save file is parsed transactionally; a malformed file is rejected without altering the live state.

Part 3 — Design Specification

3.1 High-level architecture

The codebase is organised into **9 layers** in `src/`. Each layer is a folder; each layer's headers may include only headers from layers strictly lower in the diagram below.

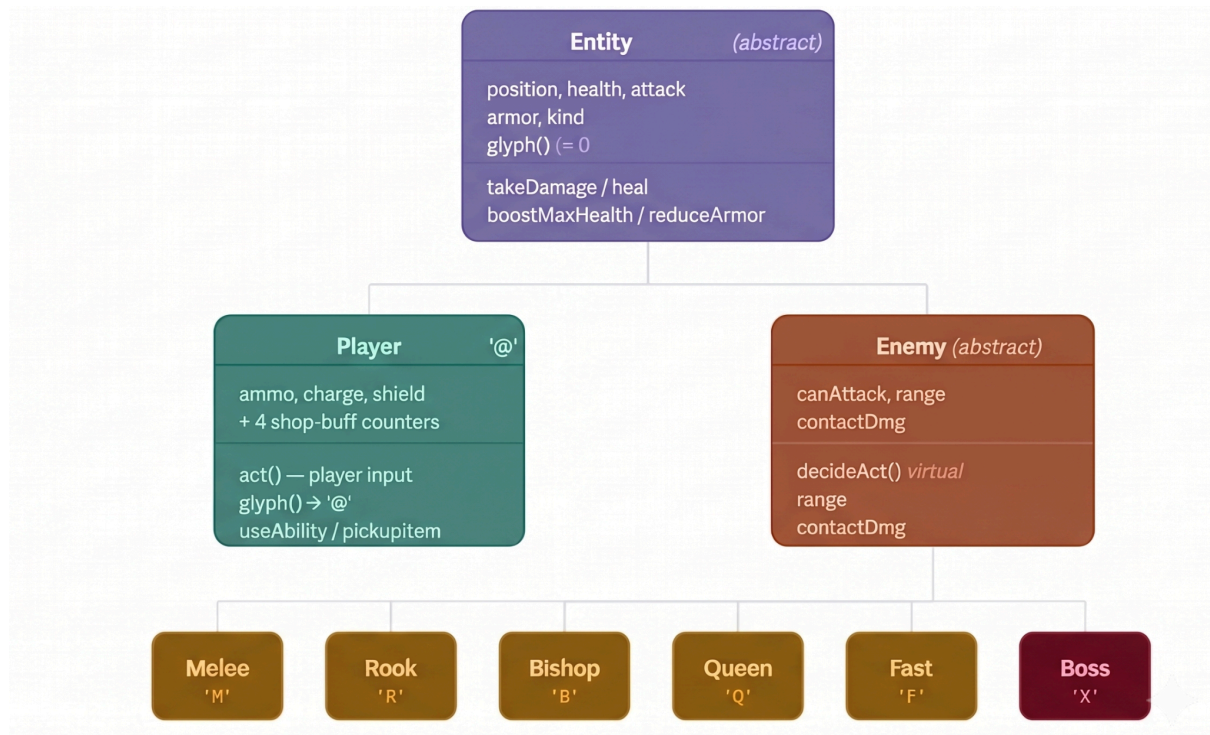


Two design rules govern this diagram:

1. **Strict downward dependency:** a header in `entities/` may include from `world/` and `core/`, but never from `systems/`, `io/`, or `render/`.
2. **Renderer talks to systems through a pure interface only:** `systems/` never includes `RaylibRenderer.h` or `ConsoleRenderer.h`. It only includes `render/IRenderer.h`. The concrete renderer header is included in `main.cpp` and nowhere else.

3.2 Class diagrams

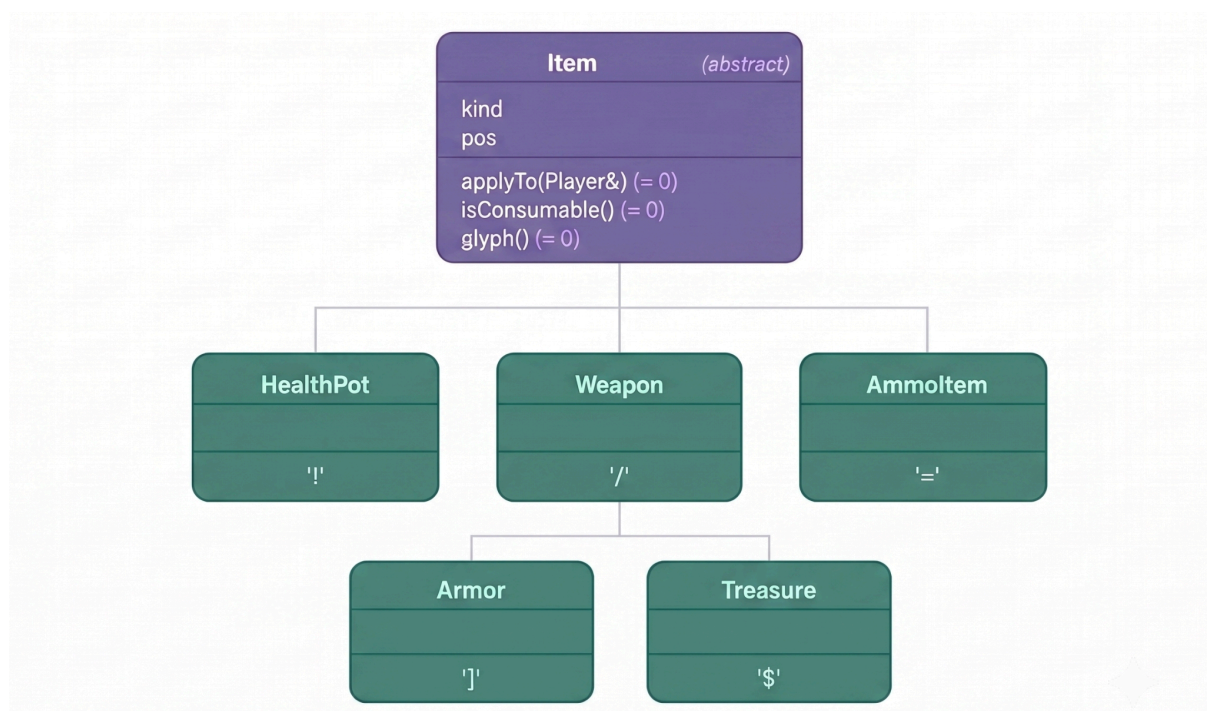
3.2.1 Entity hierarchy



Each concrete Enemy overrides:

- `canAttackPlayer (map, playerPos)` — the chess-style geometric rule.
- `glyph()` — the single character drawn on the map.
- `movesPerTurn()` — only `FastEnemy` overrides this to return 2.

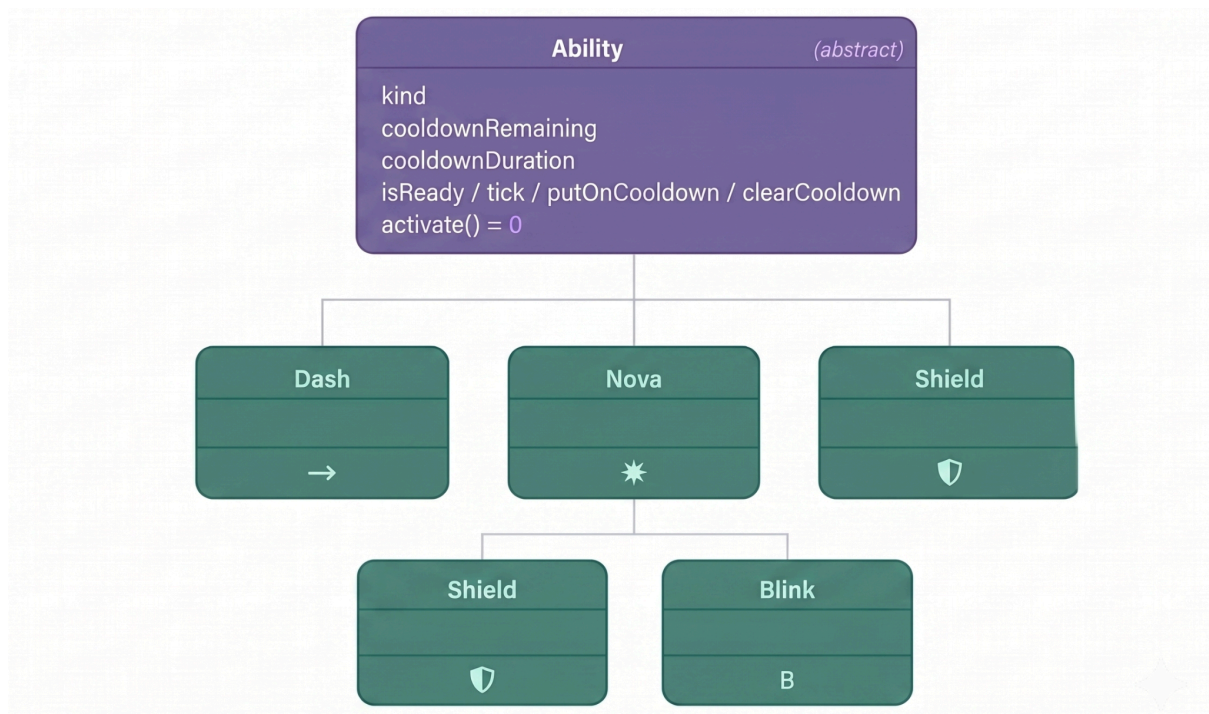
3.2.2 Item hierarchy



Notable specialisations:

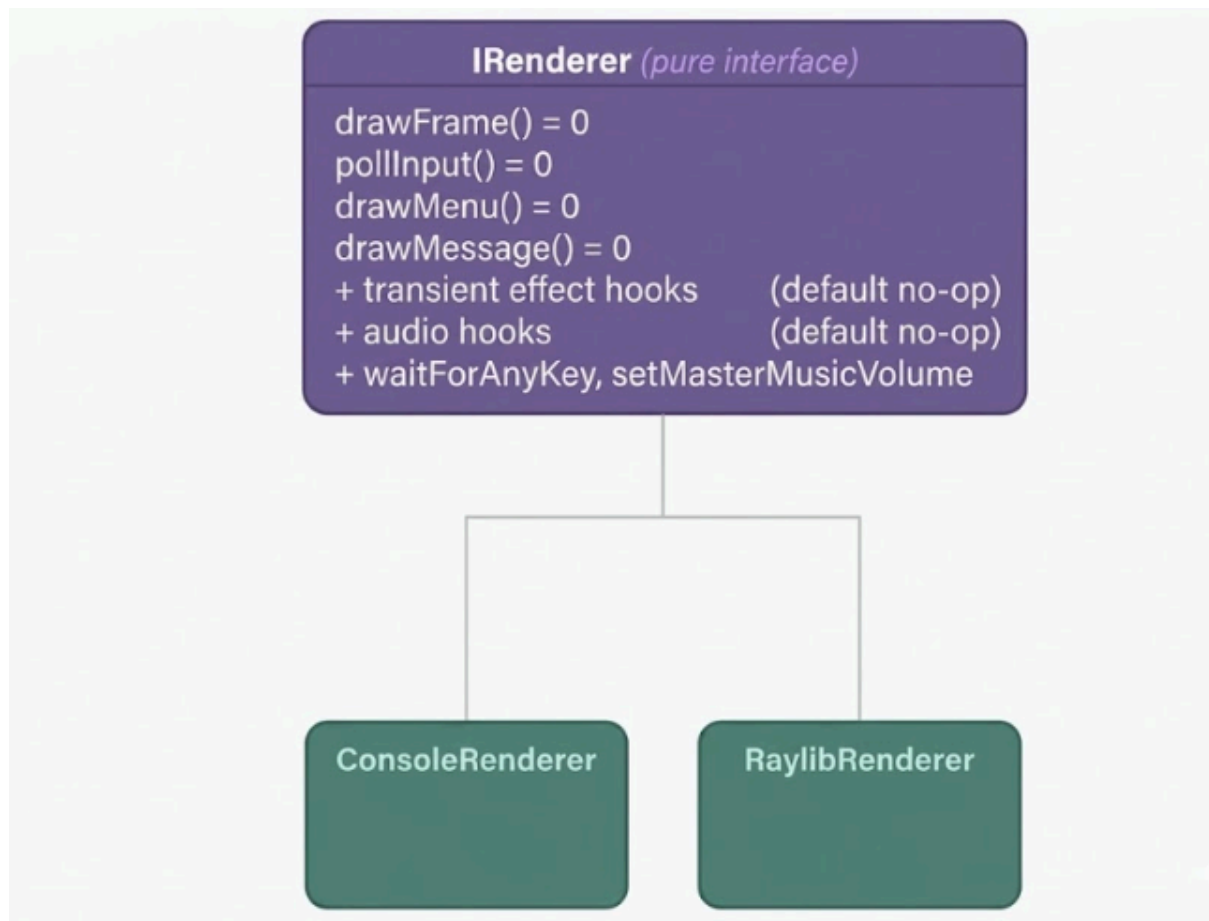
- `Weapon` is the only item with a *replacing* effect on `attack`. Picks up are guarded so a weaker weapon never reduces the player's stat.
- A `Weapon` flagged `ranged` additionally calls `Player::addFireRange` and grants the omnidirectional spread shot.
- `Treasure::applyTo` is intentionally a no-op; the credit happens in `TurnManager` through `state.addScore` and `state.addGold`.

3.2.3 Ability hierarchy



`Ability::activate()` is intentionally parameterless — concrete abilities receive their context (map, player, enemies, log) through `AbilitySystem::applyXxx` helpers because the ability itself does not own state machinery; only the cooldown bookkeeping lives on the base class.

3.2.4 Renderer hierarchy



`ConsoleRenderer` overrides only the four core methods and inherits every audio / effect hook as a no-op. It does **not** show fire trails, enemy attack beams, the Nova shockwave, the player melee slash, the Death Dungeon overlay, or play any sound. Its purpose in the project is to prove that the **'IRenderer'** abstraction is loosely coupled — the same game core runs through it unchanged. `RaylibRenderer` is the canonical implementation and overrides every hook for the full graphical experience.

3.3 Data members of major classes

3.3.1 Entity (`entities/Entity.h`)

Member	Type	Purpose
<code>position_</code>	<code>Vec2</code>	Current grid cell.
<code>health_</code>	<code>int</code>	Current HP; 0 means dead.
<code>maxHealth_</code>	<code>int</code>	Cap that <code>health</code> cannot exceed.

Member	Type	Purpose
attack_	int	Outgoing damage stat.
armor_	int	Damage absorber; consumed before HP.
kind_	EntityKind	Polymorphism tag (avoids <code>dynamic_cast</code>).

3.3.2 Player (extends Entity, `entities/Player.h`)

Member	Type	Purpose
ammo_	int	Ranged ammunition.
fireRange_	int	Cells a fired projectile can travel.
chargeMeter_	int	Nova charge, 0 to <code>chargeMeterMax</code> .
shieldRemainingTurns_	int	Shield duration counter.
fireCooldown_	int	Turns until the next Fire is allowed.
fireCooldownDuration_	int	How long the cooldown lasts on reset.
wallPierceShotsRemaining_	int	Piercer Round shop-buff charges.
doubleMoveTurnsRemaining_	int	Quickstep shop-buff charges.
blinkChainUsesRemaining_	int	Blink Chain shop-buff charges.
twinStrikeChargesRemaining_	int	Twin Strike shop-buff charges.
equippedWeapon_	Weapon* (non-owning)	Currently equipped weapon, or null.
inventory_	Inventory (by value)	Items the hero is carrying.
abilities_	vector<unique_ptr<Ability>>	Owned ability instances.

3.3.3 GameState (`systems/GameState.h`)

Member	Type	Purpose
map_	GridMap	The dungeon grid for the wave.
player_	Player	The hero.
enemies_	vector<unique_ptr<Enemy>>	Live enemies.
items_	vector<unique_ptr<Item>>	Floor items.
waveNumber_	int	Current 1-based wave index.
score_	int	Accumulated run score (leaderboard input).
gold_	int	Spendable currency for the Shop.
turnCount_	int	Turns elapsed this run.
enemiesKilled_	int	Kill count for the run.
rng_	Rng	Deterministic randomness source.

3.3.4 TurnResult (systems/TurnManager.h)

TurnManager::processTurn returns a populated TurnResult so the calling Game::playingLoop can drive renderer side-effects without re-querying state.

Member	Type	Meaning
turnConsumed	bool	Did this call advance the turn counter?
quitRequested	bool	The player asked to quit to menu.
saveRequested	bool	The player asked to save.
playerDied	bool	HP reached zero.
waveCleared	bool	All enemies defeated this turn.
fireEffect	FireResult	Trail / impact / hit data for the renderer.
enemyAttacks	vector<EnemyAttackInfo>	One entry per enemy strike for the renderer.
novaFired	bool	Nova activation flag.

Member	Type	Meaning
novaCenter, novaRadius	Vec2, int	Where to draw the shockwave.
playerMeleed	bool	The player executed a melee strike.
playerMeleeTarget	Vec2	Where to stamp the melee X.
itemPickedUp	bool	A pickup chime should play.
abilityActivated	bool	An ability went off; play its sound.
abilityKind	AbilityKind	Which ability fired.

3.3.5 RaylibRenderer (truncated, render/RaylibRenderer.h)

Group	Selected fields
Cached composition	cachedState_, cachedConfig_, cachedLog_, gameFrameActive_, pendingReset_, menuOptions_, messageLines_
Fire / nova / melee transient	fireTrailCells_, fireImpactCell_, fireTrailFramesRemaining_, novaCenter_, novaRadius_, novaFramesRemaining_, meleeEffectCell_, meleeEffectFramesRemaining_
Hell mode (Death Dungeon)	hellModeLastWaveNumber_, hellModeWaveStartTime_, hellModeActive_, hellModeLaughTriggered_, hellEmbers_
Audio	audioReady_, 12 procedural Sounds, musicStream_ (ambient pad), bossMusic_, normalMusic_, masterMusicVolume_

3.4 Function members (key signatures)

3.4.1 TurnManager

C++

```
TurnResult processTurn(GameState&, const InputCommand&,
                      CombatSystem&, AbilitySystem&,
                      const Pathfinder&, const Config&, EventLog&) const;
```

Five-phase turn loop — exhaustively documented in §3.5 below.

3.4.2 CombatSystem

C++

```
void applyAttack(Entity& attacker, Entity& target, EventLog&) const;

bool firePlayerProjectile(const GridMap&, Player&, const Vec2& dir,
                        std::vector<unique_ptr<Enemy>>&,
                        const Config&, EventLog&,
                        FireResult& outResult) const;

void resolveDeaths(std::vector<unique_ptr<Enemy>>&, Player&,
                  EventLog&, int& killCount,
                  int& chargeMeter, int chargeMeterMax,
                  bool& playerDead) const;
```

`applyAttack` doubles damage when the attacker is the player and a Twin Strike charge is active; armour absorbs first, HP receives the spill. `firePlayerProjectile` traces a Bresenham ray, walks through walls when a Piercer Round charge is active, fires 7 extra beams when the player has the omnidirectional spread shot, and sets the per-shot fire cooldown after a successful trace.

3.4.3 AbilitySystem

C++

```
bool activate(AbilityKind, const Vec2& dir,
             GameState&, const Config&, EventLog&,
             AbilityEffectInfo* outEffect = nullptr) const;

void tickCooldowns(Player&) const;

void addCharge(Player&, int amount, int maxCharge) const;
```

`activate` looks up the ability by kind, gates it on cooldown / charge, dispatches into the matching `applyXxx` helper, and finally calls

`putOnCooldown`. The Blink Chain shop-buff causes the cooldown to be cleared again immediately after activation.

3.4.4 WaveManager

C++

```
void startWave(GameState&, const Config&, int spawnCount) const;  
void advance(GameState&, const Config&) const;  
bool isWaveCleared(const GameState&) const;
```

`startWave` regenerates the map, picks spawn cells through `MapGenerator::pickSpawns`, places one boss on every fifth wave or a mixed roster on normal waves, and applies the per-wave HP scaling through `Entity::boostMaxHealth`.

3.4.5 SaveManager

C++

```
static void save(const GameState&, const std::string& filePath, EventLog&);  
static bool load(const std::string& filePath, GameState&, EventLog&);
```

`load` is the **transactional** function described above: every parsed value lands in a local staging variable; commitment to `state` happens only after the entire file is validated. The `GOLD` field is optional for backwards compatibility.

3.4.6 IRenderer

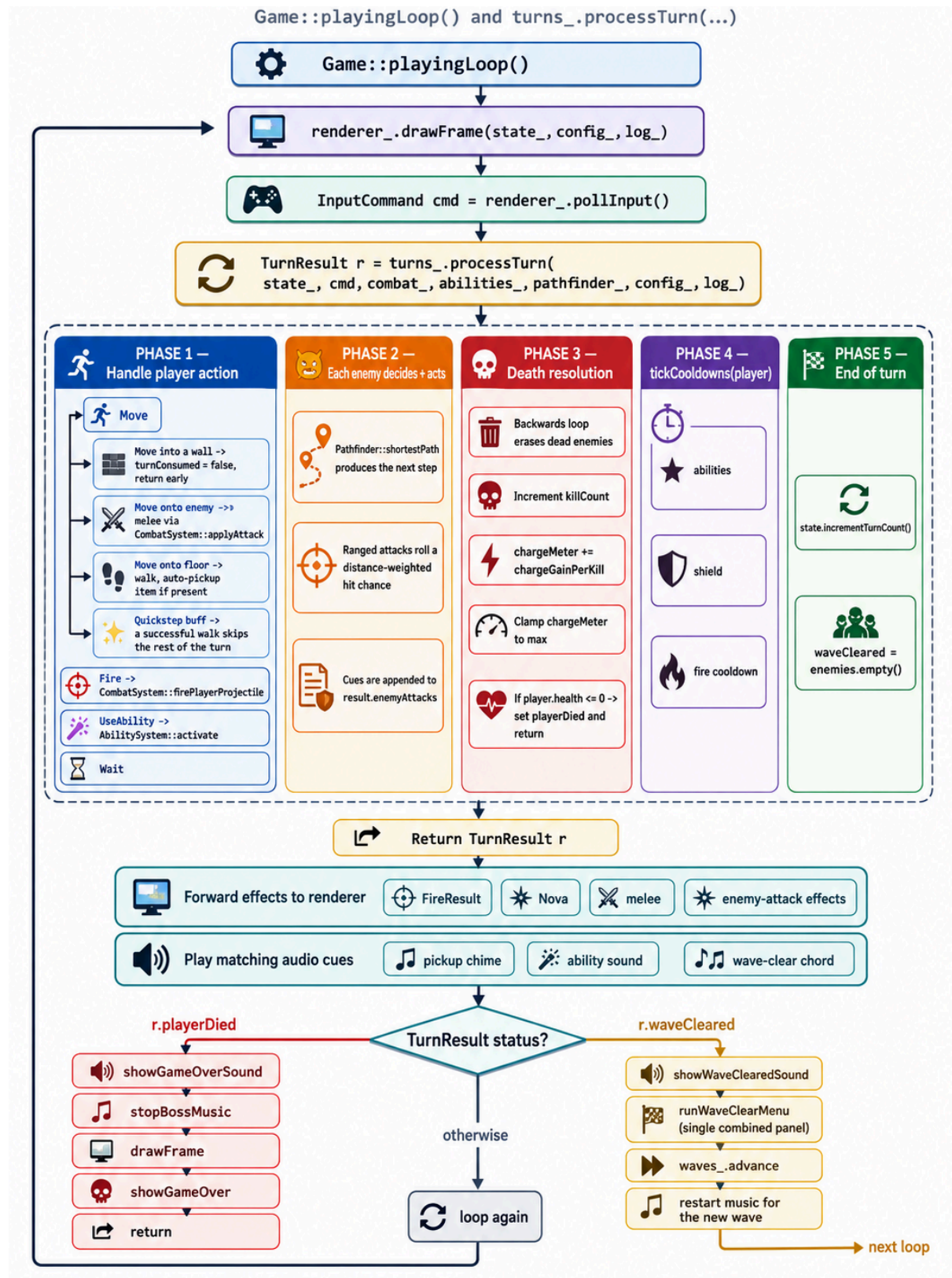
C++

```
virtual void drawFrame(const GameState&, const Config&, const EventLog&) = 0;
virtual InputCommand pollInput() = 0;
virtual void drawMenu(const std::vector<std::string>&, int selected) = 0;
virtual void drawMessage(const std::string&) = 0;

// Optional hooks (default no-op implementations):
virtual void showFireEffect(const std::vector<Vec2>&, const Vec2&, bool hit) {}
virtual void showEnemyAttackEffect(const Vec2&, const Vec2&, bool ranged, bool hit) {}
virtual void showNovaEffect(const Vec2& center, int radius) {}
virtual void showPlayerMeleeEffect(const Vec2& targetCell) {}
virtual void showPickupSound() {}
virtual void showWaveClearedSound() {}
virtual void showGameOverSound() {}
virtual void showAbilitySound(AbilityKind kind) {}
virtual void setBossMusicActive(bool active) {}
virtual void setNormalMusicActive(bool active) {}
virtual void setMasterMusicVolume(float volume) {}
virtual void waitForAnyKey() { (void)pollInput(); }
```

The default `waitForAnyKey` falls back to `pollInput` so the ASCII smoke-test build inherits a working "press anything" behaviour without overriding.

3.5 Sequence diagram — one full turn



3.6 File I/O design

3.6.1 Save format (data/save.dgs)

The save file is **plain UTF-8 text**, one tagged record per line:

```
SEED 12345
WAVE 7
SCORE 1430
GOLD 21
TURN 247
KILLS 36
PLAYER_POS 12 6
PLAYER_HEALTH 88
PLAYER_AMMO 9
PLAYER_ARMOR 4
PLAYER_ATTACK 22
CHARGE 60
MAP_WIDTH 40
MAP_HEIGHT 24
TILE 0 0 W
TILE 1 0 F
...
ENEMY Rook 18 4 24
ENEMY Boss 5 12 80
ITEM Treasure 22 11 17
ITEM Weapon 19 14 22 1
...
```

Tag → field mapping:

Tag	Fields	Notes
SEED	unsigned int	Run seed; reseeds the Rng on load.
WAVE	int	Current wave number.
SCORE	int	
GOLD	int	Optional. Missing → defaults to 0.
TURN, KILLS	int	
PLAYER_*	int(s)	Pos, health, ammo, armor, attack.
CHARGE	int	Nova charge meter.
MAP_*	int	Width / height.
TILE	int int char	x, y, F/W.
ENEMY	string int int int	Kind tag, x, y, health.
ITEM	string int int [extras]	Kind tag, x, y, kind-specific extras.

3.6.2 Load semantics

`load()` performs **staging then commit**:

1. Each parsed value goes into a *local* staging variable.
2. `bool found*` flags track whether each required field was seen.
3. After EOF, every required `found*` flag is checked.
4. If anything is missing or any line is malformed the function returns `false` **without writing a single byte to `state`**.
5. On a fully validated parse, every staged value is written into `state` in one pass.

This guarantees that a corrupt save can never partially overwrite the live run.

3.6.3 Leaderboard (`data/highscores.txt`)

The leaderboard uses the `wave × 100 + kills × 10 + treasure` formula (`ScoreBoard::computeScore`). Entries are kept sorted by descending score; new entries are inserted via insertion sort and the file is truncated to the top N (default 10).

3.7 Algorithms catalogue

Algorithm	Source file	Purpose
Breadth-first search	<code>world/Pathfinder.cpp</code>	Enemy AI step-toward-player; shortest path on grid.
Bresenham line traversal	<code>world/LineOfSight.cpp</code>	Projectile rays, fire-aim preview, Blink targets.
Drunkard's walk + flood-fill	<code>world/MapGenerator.cpp</code>	Connected dungeon generation.
Fisher-Yates partial shuffle	<code>systems/UpgradeSystem.cpp</code> , <code>systems/Shop.cpp</code>	Drawing 3 distinct items per draft.
Insertion sort	<code>io/ScoreBoard.cpp</code>	Top-N leaderboard sort on append.
Hand-rolled singly linked list	<code>systems/EventLog.cpp</code>	HUD's rolling event-message buffer.

Algorithm	Source file	Purpose
Templates <code>Grid<T></code> , <code>RingBuffer<T></code>	<code>core/Grid.h</code> , <code>core/RingBuffer.h</code>	Generic 2-D buffer / fixed-capacity FIFO.
Finite state machine	<code>systems/GameStateMachine.cpp</code>	Top-level run phase transitions.
Procedural audio synthesis	<code>render/RaylibRenderer.cpp</code> (<code>makeXxxSound</code>)	Every SFX is built in memory at startup.
AudioStream callback synthesis	<code>render/RaylibRenderer.cpp</code> (<code>audioStreamCallback</code>)	Real-time procedural ambient pad.

3.8 Coupling and encapsulation

No globals, no singletons. Every piece of run state is owned by one `GameState` instance constructed in `main.cpp`. Systems receive it by reference and never store a copy.

Encapsulation: every data member of every class is `private` or `protected`. Public access goes through accessor / mutator pairs. The mutators clamp inputs (`heal` clamps to `maxHealth`, `addX` rejects negative values, `spendGold` clamps the wallet at zero) so inappropriate calls cannot drive the state into an invalid configuration.

Renderer abstraction. `IRenderer` is a pure interface. The two concrete renderers compile in parallel; the active one is chosen at build time by the `-DDGA_WITH_RAYLIB` flag in `main.cpp`. Other systems include only `render/IRenderer.h`. The Raylib build is the only one that links against `raylib.lib`.

3.9 Module division summary

Each folder under `src/` has a single, focused responsibility. Reusable helpers live in `core/` and never reach upward. The table below describes each module's purpose.

Module	Responsibility (single, focused)
<code>core</code>	Foundational types and utilities used by every other layer.
<code>world</code>	Static geometry: grid, tiles, line of sight, pathfinding, generation.

Module	Responsibility (single, focused)
entities	Living actors and their AI hooks.
items	Floor pickups and the player's inventory.
combat	Damage formula, projectile trace, melee resolution.
abilities	Cooldown-gated player powers (Dash / Nova / Shield / Blink).
systems	Whole-game orchestration: turn loop, wave progression, draft, shop, state machine, event log.
io	File-backed persistence: transactional save / load and the leaderboard.
render	Output and input abstraction. Concrete implementations are pluggable.

3.10 Risks and design trade-offs

- Trade-off (RaylibRenderer file size).** `render/RaylibRenderer.cpp` is the largest source file in the project (it owns synth helpers, HUD layout, effect rendering, hell-mode overlay, and audio toggling). Splitting it into 3 – 4 smaller translation units would reduce per-file size but introduce more inter-file friction. We kept it monolithic for the submission and noted the split as future work in the Summary Report.
- Risk (forward-declaration cycles).** Any new owning relationship between `Entity / Item / Player / Enemy` must keep destructors out-of-line, lest the owning header demand a complete type for the forward-declared class. The cycle bites once and never again, but the discipline must be applied consistently.
- Trade-off (insertion-sort leaderboard).** $O(N)$ per append, which is trivial for top-10. No need for a heap or balanced tree at this scale. The simplicity wins.
- Trade-off (procedural SFX vs. asset files).** Every sound effect is synthesised at startup from sine / noise samples. The trade is startup cost (≈ 200 ms) for a zero-asset shipping size and the ability to retune any sound by editing one constant. For a music track the trade flips: the OGG streaming path makes the boss / normal themes feel like real music, which is impossible to fake procedurally at the same quality budget.
- Trade-off (manual `[[maybe_unused]]` versus deletion).** Some sound-synthesis helpers (`makeNoiseBurst`, `rotateLeft45`, `rotateRight45`) are no longer actively called after the spread shot was promoted to omnidirectional and the noise-burst SFX were superseded by `makeNoiseAndTone`. They remain in the file marked `[[maybe_unused]]` because they are part of the documented audio

toolkit and are likely to be re-used by a future SFX. Deleting them would shrink the file by a few hundred lines but lose the documentation context.

Conclusion

This Final Report combines the Project Proposal, Requirements Analysis, and Design Specification into the single deliverable required by the course. The companion documents — the User Manual (operational documentation), the Summary Report (project experience and AI usage), and the test-suite README (testing methodology) — fill out the remaining mandatory items. The submission is complete to the level of quality demonstrated above and the running game and tests are reproducible from the source archive on any C++17-capable compiler.